

Bases de données
documentaires et distribuées,
<http://b3d.bdpedia.fr>

Streaming avec Flink

Le *Streaming* avec Flink

On a affaire à un **stream**, flux de données de taille (conceptuellement) infinie.

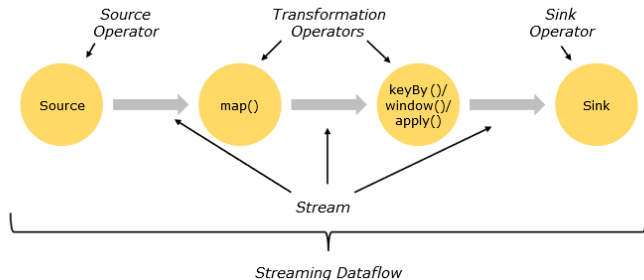
Les éléments de ce flux sont des **items** dans la terminologie Flink. Un item est de type quelconque, on doit parfois lui attribuer une clé de regroupement.

Et on applique à ce flux et à ces items des **transformations**, en continu

Contrairement à Spark, Flink traite nativement les flux : les items sont pris en compte un par un, instantanément (alors que Spark fait du micro-batch, ou pseudo-streaming)

Les *workflow* de Flink

La source est typiquement un gestionnaire de flux comme Kafka.



La cible est typiquement une base de données.

Les opérateurs (voir la liste dans support de cours) sont à peu près ceux que l'on trouve dans Pig ou Spark.

Notre simulateur de flux

Pour expérimenter facilement, nous utilisons un simulateur de flux écrit en Python, `Simflux.py`.

- Pour engendrer un flux d'entiers sur le port 9000 (par défaut) :
`python3 SimFlux.py`
- Pour engendrer un flux de lignes extraites d'un fichier texte :
`python3 SimFlux.py --source nom_fichier`
- Pour engendrer un flux des contenus de fichier d'un répertoire :
`python3 SimFlux.py --source répertoire`

On va tester la façon dont Flink consomme ce flux en entrée.

Un premier exemple avec l'interpréteur Scala

Au préalable, lancer notre générateur de flux sur le port 9000.

```
// Déclaration du flux de données. NB: senv est l'environnement de streaming  
val stream = senv.socketTextStream("localhost", 9000, '\n')  
// À chaque entier en entrée, on ajoute 2 (pourquoi pas)  
val w = stream.map ({ x => x.toInt + 2 } )  
// Voyons ce que cela donne  
w.print()  
// Workflow défini: on l'exécute  
senv.execute("Mon premier traitement de flux ")
```

Notez bien que l'exécution est “paresseuse” : on ne déclenche le *workflow* que quand le *sink* est défini.

Un workflow = séquence de transformations

S'exprime très simplement en Scala.

Ci-dessous : on crée un Tuple scala avec l'entier reçu, puis on lui associe son double, puis on affiche.

```
stream.map ( { x => Tuple1(x.toInt) } )  
      .map( {y => (y._1, y._1 * 2) } )  
      .print()
```

Astuce : utiliser :paste puis CTRL D pour entrer des commandes multi-lignes.

Traitons des flux d'objets

Plus intéressant : on transforme le flux en entrée en un type donné (comme dans Pig)

```
case class MonDouble(leReel: Float, sonDouble: Double)
val stream = senv.socketTextStream("localhost", 9000, '\n')
val w = stream.map ( { x => Tuple1(x.toFloat) } )
               .map( {y => MonDouble(y._1, y._1 * 2) } )
val fluxFiltre = w.filter ({_.leReel > 1000})
fluxFiltre.print()
senv.execute("Mon premier traitement de flux ")
```

Important : *Streaming* = pipelining. **Il n'y a pas de matérialisation des résultats intermédiaires.**

Passons à des chaînes de caractères

Ci-dessous : après avoir lancé le simulateur en mode 'lecture de fichier texte', on traite les lignes de texte constituant le flux.

```
val stream = senv.socketTextStream("localhost", 9000, '\n')
val w = stream.flatMap ({ str => str.split("\\W+") }).print()
senv.execute("Je découpe en mots")
```

Nous sommes prêts à compter les mots.

Le fameux compteur de mots

Voici la chaîne de traitement pour compter les mots d'un flux en entrée.

```
case class CompteurMot(mot: String, compteur: Int)
val stream = env.socketTextStream("localhost", 9000, '\n')
val w = stream.flatMap({ str => str.split("\\W+") })
    .map({ CompteurMot(_, 1) })
    .keyBy("mot")
    .sum("compteur")
    .print()
env.execute("Le compteur de mots")
```

Notez. (i) l'opérateur keyBy qui regroupe des nuplets Scala sur un champ (ici, le champ "mot") et crée donc autant de flux que de mots; (ii) le compteur ne fait que s'incrémenter.

Le même, en utilisant reduce

Que signifie reduce dans un contexte de flux ? C'est l'accumulation successive des valeurs rencontrées, groupées d'après une clé.

```
case class CompteurMot(mot: String, compteur: Int)
val stream = senv.socketTextStream("localhost", 9000, '\n')
val mots = stream.flatMap({ str => str.split("\\W+") })
                  .map({ CompteurMot(_, 1) }).keyBy("mot")
val compte = mots.reduce( (acc, occ)
    => {CompteurMot (acc.mot, acc.compteur + 1) }).print()
senv.execute("Le compteur de mots")
```

Et voilà ! Nous savons traiter des flux *infinis*. Il reste à savoir les découper en **fenêtres**.